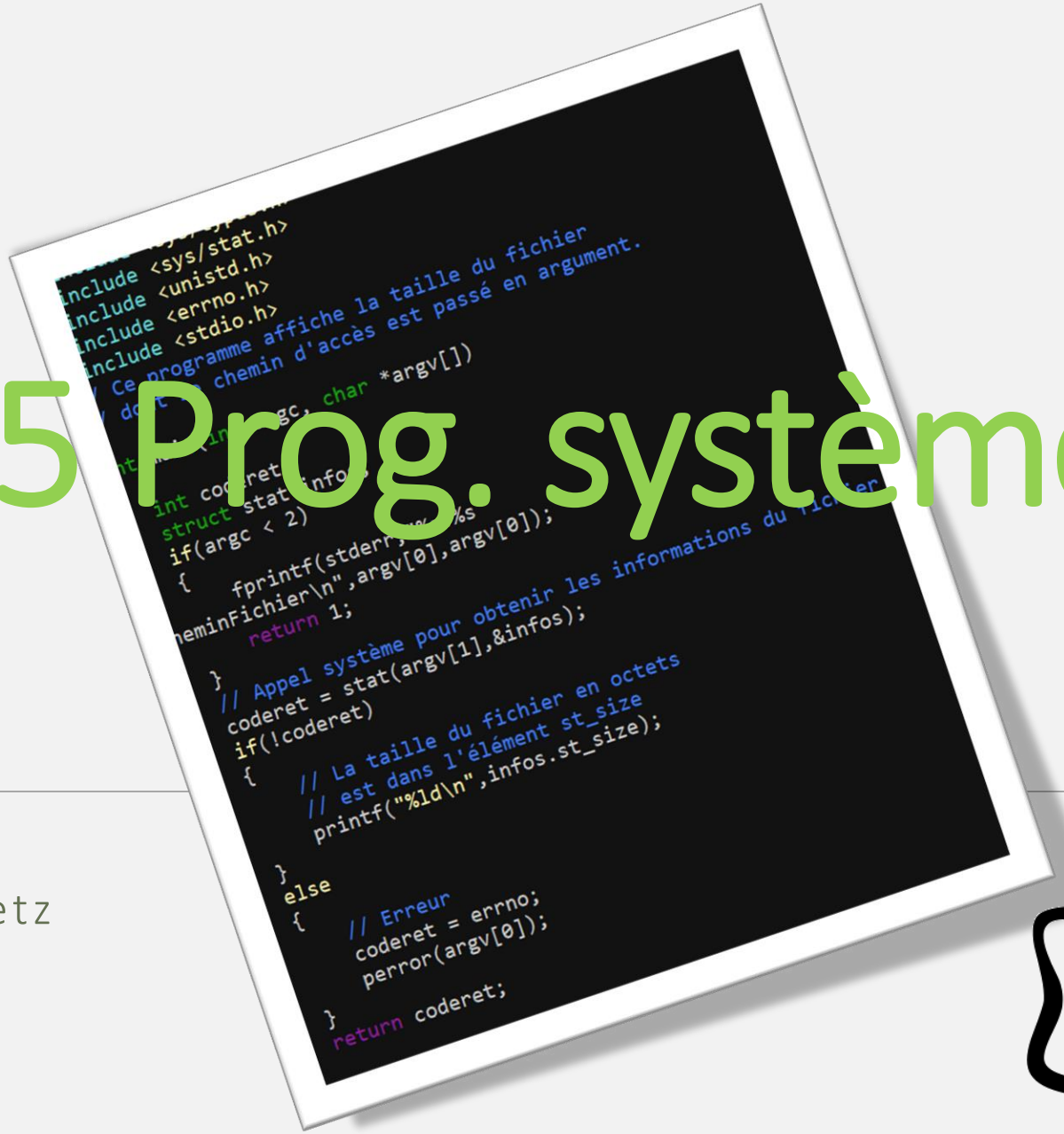
R3.05 Prog. système



Romain JAGER

IUT Informatique – Metz

Bureau B35

Bonnes pratiques en C

```c
#include <stdio.h>

int main(int argc, char** argv) {
  int i;
  for(i=1;i<argc;i++) print("%s\n",argv[i]);
  return 0;
}
```

- Attention, les programmes sont en "pur C", et n'utilisent pas certaine facilités du C++. Ceci implique que vous devez créer vos exécutables en utilisant gcc et non pas g++.
- Il existe une fonction particulière par laquelle commence l'exécution d'un programme. En C et Java, cette fonction s'appelle main()
- Comme toute fonction, on peut lui passer des paramètres, qu'on appelle généralement les paramètres (ou arguments) d'un programme
- Dans la plupart des langages, la fonction main() reçoit ces paramètres sous la forme d'un tableau de chaîne de caractères, sachant que le type chaîne prend différentes formes selon le langage
- Cependant le type chaîne de caractères n'existe pas vraiment en C et l'on recourt à un une "astuce" : un tableau d'octets dont le dernier vaut 0
- en C, il faut se dire que 1 char = 1 octet
- En C, on déclare un tableau d'octets nommé tab avec char tab[...] (les ... sont la taille)
  ou bien char* tab
- Donc , pour avoir un tableau de tableaux de char, on note char tab[...][...] ou bien char**. C'est ce qui est utilisé en C.

Accès aux paramètres

- en C, les tableaux ne sont pas gérés comme en Java, notamment sur le fait qu'on ne peut pas connaître leur taille, à moins d'avoir une autre variable qui l'indique
- c'est pourquoi en C, main() a un 2ème paramètre, argc, qui indique la taille du tableau de chaînes donc le nombre de paramètres
- en C, ce tableau contient en première case le nom du programme. C'est pourquoi pour afficher uniquement les paramètres, on commence à l'indice 1

```
#include <stdio.h>

int main(int argc, char** argv) {
  int i;
  for(i=1;i<argc;i++) print("%s\n",argv[i]);
  return 0;
}
```

Test des paramètres

- tester le nombre de paramètres est obligatoire si on veut limiter les failles potentielles de sécurité
- Le minimum est donc de faire des tests sur argc et vérifier s'il correspond aux besoins du programme. Si ce n'est pas le cas, on appelle une fonction qui précise l'usage du programme
- le mieux étant de faire des tests sur le contenu des paramètres, notamment leur longueur ou leur format quand il s'agit de chaîne ou bien sur leur valeur
- Exemple : un programme qui prend en paramètre une adresse IP et un n° de port.
  Quand on représente une adresse IP sous la forme d'une chaîne de caractères, c'est un quadruplet séparé par des points, par exemple 123.45.67.89. Chaque membre de ce quadruplet a une valeur entre 0 et 255. Donc en terme de longueur de chaîne, le minimum est 7 (pour A.B.C.D) et le maximum 15 (AAA.BBB.CCC.DDD). Toute valeur inférieure à 7 et supérieure à 15 veut dire que l'IP est invalide
- Pour le n° de port, sa valeur doit être comprise entre 1 et 65535 (NB : c'est un type short sur 2 octets). Toute valeur supérieure ou inférieure est invalide.

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

void usage() {
   fprintf(stderr,"usage : myprog ip port\n\t- ip : A.B.C.D (with 0<A,B,C,D<255)\n\t- port : from 0 to 65535\n");
   exit(1); // termine l'exécution
}

int main(int argc, char**argv) {
   if (argc != 3) usage();
   if ((strlen(argv[1]) < 7) || (strlen(argv[1]) > 15)) usage();
   int port = atoi(argv[2]);
   if ((port < 0) || (port > 65535)) usage();
   printf("ip : %s, port : %d\n",argv[1],port);
   return 0;
}
```

```bash
gcc -c exo1.c
gcc exo1.c -o exo1
```

Lancez l'exécutable avec les lignes suivantes :

- exo1
- exo1 1.2.3.4
- exo1 1.2.3.4 -10
- exo1 1.2.3.4 21
- exo1 1.2.3.4.5.6 21

Test des paramètres

- On remarque que la dernière instruction ne provoque pas d'appel à usage(), ce qui est normal car le premier paramètre a bien une longueur entre 7 et 15.
- Pour autant, l'adresse ip indiquée est invalide. Il faudrait donc tester sa structure en plus de sa taille, et vérifier la présence stricte d'un quadruplet avec des valuers entre 0 et 255.
- Malheureusement, en C, il n'y a pas de moyen très simple de tester le format d'une chaîne.
- Par exemple, on peut d'abord utiliser `sscanf()` pour affecter des valeurs à des variables en prenant ces valeurs dans une chaîne. Ensuite, il suffit de tester ces valeurs.
- Dans le cas présent, cela revient à trouver dans `argv[1]` 4 valeurs entières séparées par des points et de les affecter à 4 int différents.

Test des paramètres

On modifie donc le code comme suivant :

```
...
int main(int argc, char**argv) {
   ...
   int p1,p2,p3,p4;
   int nbOk = sscanf(argv[1],"%d.%d.%d.%d",&p1,&p2,&p3,&p4);
   if (nbOk != 4) usage();
   if ((p1<0)||(p1>255)||(p2<0)||(p2>255)||(p3<0)||(p3>255)||(p4<0)||(p4>255)) usage();

   printf("ip : %d.%d.%d.%d, port : %d\n",p1,p2,p3,p4,port);
   return 0;
}
```

Compilation avec l'option -Wall

```c
#include <stdio.h>

void afficheDate(int day, int month, int year) {
  if (year = 0) return;
  printf("Nous somme le %d/%d/%d\n",day,month,year);
}

int main(int argc, char** argv) {

  int day,month,year;
  afficheDate(day,month,year);
}
```

- Copiez/collez le programme ci-dessus dans un éditeur et sauvegardez-le sous le nom exo2.c
- Créez l'exécutable, d'abord en compilant puis en faisant l'édition de lien

```
gcc -c exo2.c
gcc exo2.o -o exo2
```

- Vous remarquez que la compilation ne produit aucun warning.
- Si on exécute le programme, il affiche un message du style : Nous sommes le 0/0/0
- En examinant le source, on s'aperçoit que day, month et year ne sont pas initialisées. D'après l'affichage, Il semble que leur valeur par défaut soit 0. Est-ce vraiment le cas ?

Compilation avec l'option -Wall

- Modifiez le programme comme suivant :

```
...
int main(int argc, char** argv) {
  int i=0;
  int day,month,year;
  afficheDate(day,month,year);
```

- Recréez l'exécutable et lancez-le. Cette fois, on obtient un résultat différent.
  Par exemple : Nous sommes le 0/32767/0
- On remarque donc que le compilateur ne donne pas de valeur par défaut aux variables, puisqu'elles ne sont toujours pas initialisées et pourtant la valeur d'une d'entre-elles à changé par rapport au premier test.
- ***Question 1*** : comment est-il possible que l'ajout d'un simple variable (i en l'occurence) dans le source puisse modifier la valeur que contient les variables non initialisées ?
  - Recompilez en forçant l'affichage de tous les warnings (option -Wall) :

```
gcc -Wall -c exo2.c
```

Compilation avec l'option -Wall

Le résultat est le suivant :

```
user1@iutm-inf-roja-p:~/R305/basesC$ gcc -Wall -c exo2.c
exo2.c: In function 'afficheDate':
exo2.c:4:7: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
    4 |   if (year = 0) return;
      |       ^~~~
exo2.c: In function 'main':
exo2.c:10:7: warning: unused variable 'i' [-Wunused-variable]
   10 |   int i=0;
      |       ^
exo2.c:12:3: warning: 'day' is used uninitialized in this function [-Wuninitialized]
   12 |   afficheDate(day,month,year);
      |   ^~~~~~~~~~~~~~~~~~~~~~~~~~~
exo2.c:12:3: warning: 'month' is used uninitialized in this function [-Wuninitialized]
exo2.c:12:3: warning: 'year' is used uninitialized in this function [-Wuninitialized]
user1@iutm-inf-roja-p:~/R305/basesC$
```

Les 3 derniers warnings indiquent clairement un problème potentiel : certaines variables ne sont pas initialisées !

·Le premier warning est un peu moins clair : il signale qu'il y a une affectation entre parenthèses et que la valeur affectée sert pour le if.

·Dans le cas présent, la valeur est 0, donc c'est comme si on écrivait if (0) ... c'est-à-dire un test toujours faux. Néanmoins, on a quand même affecté 0 à year.

·Bien entendu, le but de cette ligne est de tester si year vaut 0 et pas de lui affecter la valeur 0. Il faudrait donc normalement écrire if (year == 0).

Conclusion : l'option -Wall permet de signaler toutes les sources potentielles de problème et donc DOIT être utilisée.

Compilation avec l'option -Wall

Le code corrigé donne :

```
user1@iutm-inf-roja-p:~/R305/basesC$ gcc -Wall -c exo2.c
exo2.c: In function 'afficheDate':
exo2.c:4:7: warning: suggest parentheses around assignment used as truth value [-Wparentheses]
    4 |   if (year = 0) return;
      |       ^~~~
exo2.c: In function 'main':
exo2.c:10:7: warning: unused variable 'i' [-Wunused-variable]
   10 |   int i=0;
      |       ^
exo2.c:12:3: warning: 'day' is used uninitialized in this function [-Wuninitialized]
   12 |   afficheDate(day,month,year);
      |   ^~~~~~~~~~~~~~~~~~~~~~~~~~~
exo2.c:12:3: warning: 'month' is used uninitialized in this function [-Wuninitialized]
exo2.c:12:3: warning: 'year' is used uninitialized in this function [-Wuninitialized]
user1@iutm-inf-roja-p:~/R305/basesC$
```

Les 3 derniers warnings indiquent clairement un problème potentiel : certaines variables ne sont pas initialisées !

·Le premier warning est un peu moins clair : il signale qu'il y a une affectation entre parenthèses et que la valeur affectée sert pour le if.

·Dans le cas présent, la valeur est 0, donc c'est comme si on écrivait if (0) ... c'est-à-dire un test toujours faux. Néanmoins, on a quand même affecté 0 à year.

·Bien entendu, le but de cette ligne est de tester si year vaut 0 et pas de lui affecter la valeur 0. Il faudrait donc normalement écrire if (year == 0).

Conclusion : l'option -Wall permet de signaler toutes les sources potentielles de problème et donc DOIT être utilisée.

Corriger TOUS les warnings

Dans l'exemple précédent, la compilation produit trois types de warnings :

- les affectations lors de tests,
- les variables non initialisées,
- la fonction main() qui se termine sans return.
- Le code corrigé donne :

```
#include <stdio.h>
void afficheDate(int day, int month, int year) {
  if (year == 0) return;
  printf("Nous somme le %d/%d/%d\n",day,month,year);
}

int main(int argc, char** argv) {
  int day=15,month=11,year=2022;
  afficheDate(day,month,year);
  return 0;
}
```

- Compilez le code : gcc -c -Wall exo2.c. A présent, la compilation ne donne aucun warning.
- Faites l'édition de liens : gcc exo2.o -o exo2. Exécutez exo2.

Notion d'appel système > Noyau Linux

Ils permettent aux programmes de demander au **noyau** d'exécuter des opérations sur les **objets** qu'il gère (périphériques, mémoire, processeurs, réseau, etc.) et auxquels ils n'ont pas directement accès.

Le noyau Linux est chargé de :

- La gestion du ou des microprocesseurs : il donne accès au(x) processeur(s) aux différents programmes, en fonction de leur priorité, afin que tous puissent y accéder et qu'un programme ne puisse pas monopoliser le ou les processeurs (**multitâche préemptif**).
- La gestion des périphériques : en coopération avec les contrôleurs de périphériques (*drivers*), il gère les différents périphériques matériels (*devices*) et traite les demandes des programmes qui veulent y accéder.
- La gestion de la mémoire physique et virtuelle : il détermine le partage de la mémoire entre lui-même et les programmes et gère un mécanisme de mémoire virtuelle permettant d'allouer davantage de mémoire que la machine n'en possède physiquement.

Notion d'appel système > Noyau Linux

Le noyau Linux est chargé de :

- La gestion du stockage : il gère l'ensemble des périphériques de stockage (avec des pilotes de périphériques et des pilotes de systèmes de fichiers) et présente aux programmes une interface d'accès unique, sous forme d'une arborescence de répertoires et de fichiers, construite à partir de différents systèmes de fichiers.
- La gestion du réseau : il prend en charge la communication sur les réseaux auxquels participe la machine, via les cartes d'interface réseau et leurs pilotes, et permet aux différents programmes d'émettre et de recevoir des messages.
- La gestion des processus : il assure le chargement, l'exécution, le contrôle et la terminaison des programmes, sous forme de **processus**, leur permettant de s'exécuter en apparente ou en réelle simultanéité (selon les capacités processeur de la machine). Il gère la sécurité par le contrôle d'accès aux ressources en fonction des habilitations des processus et par l'isolation des zones mémoires qui leur sont allouées.
- La gestion du contrôle d'accès aux ressources : avec un modèle de sécurité basé sur les notions de compte utilisateur et de groupe, il fournit un mécanisme permettant de définir et de contrôler les permissions d'accès aux différentes ressources qu'il gère.

Appel système

Pour utiliser des ressources gérées par le noyau, un programme dispose d'une interface lui permettant de demander au noyau d'effectuer différentes opérations. Cette interface est appelée : **interface d'appels système**.

Comment s'exécute un appel système ?

- Quand un programme a besoin d'accéder à une ressource ou d'utiliser l'une des fonctionnalités du noyau, il fait, directement ou indirectement, un appel à ce dernier, en lui passant des paramètres si nécessaire, puis il se met en attente. Le noyau prend en compte l'appel, récupère les éventuels paramètres et le traite. Après exécution, il met le code retour ainsi que d'éventuelles informations résultantes à la disposition du programme appelant et le réactive selon sa priorité.
- Le programme reprend alors son exécution, peut récupérer le code retour pour déterminer si l'appel a réussi ainsi que les informations complémentaires éventuellement fournies par le noyau.

Appel système

Mode utilisateur et mode noyau

La plupart des processeurs peuvent fonctionner en deux modes distincts :

- **Le mode utilisateur**

  L'accès à la mémoire est limité à la mémoire virtuelle allouée au programme en cours d'exécution, certaines instructions processeur ne sont pas autorisées.

- **Le mode noyau**

  Les programmes s'exécutant dans ce mode (typiquement le noyau et les pilotes de périphériques) ont accès à toute la mémoire physique, au matériel, ainsi qu'à l'ensemble des instructions processeur.

Bien qu'identifiés par le noyau Linux grâce à leur numéro unique, les appels système sont désignés par un nom dans les documentations.

Exemple

L'appel système permettant de demander l'ouverture d'un fichier par le noyau Linux a comme numéro unique 2, son nom est open.

Paramètres et type de retour de la fonction main()

La fonction `main()` d'un programme C est la première fonction exécutée lors du lancement du programme. Le langage C spécifie un prototype complet pour la fonction `main()` (paramètres et un type de retour) qui, justement, permet au programme de récupérer ses paramètres.

Le prototype complet de la fonction `main()` d'un programme C est :

```
int main(int argc, char *argv[]) ;
```

où :

- `int argc` est le nombre de paramètres effectivement passé au programme. Ce nombre est toujours > 1 car le premier paramètre est toujours le nom de l'exécutable.
- `char * argv []` est un tableau de chaine de caractères contenant les paramètres effectivement passés au programme.

Le premier élément de ce tableau (`argv[0]`) contient toujours le nom du programme (ou plus précisément le chemin utilisé pour accéder au fichier exécutable). C'est bien utile car on ne sait pas, dans le code C, comment le fichier exécutable s'appelle : ce fichier exécutable a pu être renommé par exemple après avoir été compilé !
Le $i^{ème}$ élément du tableau, `argv[i]`, est une chaine qui contient le $i^{ème}$ paramètre passé au programme.

```
user1@iutm-inf-roja-p:~/R305$ gcc prg.c -o prg
user1@iutm-inf-roja-p:~/R305$ ls
gpid  prg  prg.c
user1@iutm-inf-roja-p:~/R305$ ./prg
Nombre d'arguments passes au programme : 1
 argv[0] : './prg'
user1@iutm-inf-roja-p:~/R305$ ./prg un deux trois quatre
Nombre d'arguments passes au programme : 5
 argv[0] : './prg'
 argv[1] : 'un'
 argv[2] : 'deux'
 argv[3] : 'trois'
 argv[4] : 'quatre'
user1@iutm-inf-roja-p:~/R305$
```

Paramètres et type de retour de la fonction main()

Par ailleurs, conformément au prototype de la fonction `main()`, un programme C doit retourner (`return`) lorsqu'il se termine « code de retour » entier. Ce code de retour permet au programme de faire savoir au système si une erreur a été détectée.

On retiendra que, par convention :

- le `main()` doit retourner 0 (ou la constante `EXIT_SUCCESS`, définie dans `stdlib.h`, qui vaut 0) si tout s'est bien passé
- le `main()` doit retourner un nombre **non nul** en cas d'erreur.

Le choix d'une valeur quelconque non nulle en cas d'erreur permet à votre programme de savoir quel est le type d'erreur, en associant une valeur entière à une erreur donnée. La constante `EXIT_FAILURE`, définie dans `stdlib.h`, non nulle, permet de retourner un code d'erreur par défaut si on a pas besoin de valeurs spéciales.

Toutefois, les codes de retour compris entre 126 et 255 sont réservés par le système pour des erreurs spéciales. Par exemple, un code de retour valant 139 signifie qu'une « *segmentation fault* » (erreur d'accès mémoire) a eu lieu dans le programme et que le programme a planté.

Paramètres et type de retour de la fonction main()

Le programme simple suivant affiche les paramètres qui lui sont passés depuis le Terminal :

```
//prg.c
int main ( int argc , char * argv[] ) {
   int i;
   // affichage des arguments
   printf("Nombre  d'arguments  passes  au  programme  :  %d\n",
argc);
   for(i = 0 ; i< argc ; i ++) {
      printf("   argv[%d] : '%s'\n", i, argv[i]);
   }
   return 0 ;
}
```

Une fois ce programme compilé, si l'utilisateur le lance depuis le Terminal en tapant :

```
$ ./prg.exe test.txt 2
```

alors le programme affichera dans le Terminal :

```
Nombre d'arguments passes au programme : 3
   argv[0] : './prg.exe'
   argv[1] : 'test.txt'
   argv[2] : '2'
```

puis retournera 0 (tout s'est bien passé).

La fonction exit() du C

En C, la fonction exit(int status) permet de terminer le programme en retournant le code erreur status.
Voir man exit –S3

Ainsi, ou que l'on soit dans la code, on peut utiliser exit() pour arrêter le programme. Par exemple :

```
long fact( long n) {
 if( n < 0 ) {
       exit( EXIT_FAILURE) ;
 }
 // …
}
```

Détecter le code de retour d'un programme depuis le Terminal

Pour information, puisqu'un programme C retourne un code erreur, le Terminal est bien sur armé pour traiter ce code erreur. Il suffit de savoir deux choses :

- La commande `echo` du Terminal permet d'afficher une chaine de caractère. Par exemple `echo toto` affiche toto.
- A tout instant, la variable `$?` contient toujours la valeur de retour du dernier programme exécuté

Ainsi, si on tente de supprimer un fichier toto qui n'existe pas avec rm, rm affiche un message d'erreur et retourne bien un code erreur non nul :

```
$ rm toto
rm: toto: No such file or directory
$ echo $?
1
```

```
user1@iutm-inf-roja-p:~/R305$ echo toto
toto
user1@iutm-inf-roja-p:~/R305$ echo $?
0
user1@iutm-inf-roja-p:~/R305$ rm toto
rm: cannot remove 'toto': No such file or directory
user1@iutm-inf-roja-p:~/R305$ echo $?
1
user1@iutm-inf-roja-p:~/R305$
```

Parsing des paramètres

Un programme commence par analyser (ou « *parser* ») les paramètres pour 1/ vérifier qu'ils respectent bien ce qui est attendu (qu'on a passé un entier si on attend un entier par exemple) et 2/ initialiser ses variables. L'exemple suivant propose une structure pour se faire, qui est très courante et que vous pourrez réutiliser.

Supposons qu'on veuille écrire un programme qui affiche plusieurs fois dans le Terminal le contenu d'un fichier, texte ou binaire dont le chemin est passé en paramètre.

Le format qu'on retient pour les paramètres à passer au programme est donc :

- Premier paramètre : option –txt ou –bin
- Second paramètre : nom du fichier, chaine de caractère
- Troisième paramètre : entier, optionnel

Utilisation des appels système en langage C

La fonction syscall()

Syntaxe

```
#include <sys/syscall.h>
syscall(int, ...)
```

Cette fonction permet de demander directement l'exécution d'un appel système, en spécifiant son numéro et les éventuels paramètres nécessaires.

Syscall > exemple

Exemple

Ce programme, gpid.c, affiche le numéro du processus exécutant le programme, en utilisant directement l'appel système getpid() (son numéro, 39, est associé à la constante symbolique SYS_getpid).

```
#include <unistd.h>
#include <sys/syscall.h>
#include <sys/types.h>
#include <stdio.h>
int main(int argc, char *argv[])
{
   pid_t mypid;
   mypid = syscall(SYS_getpid);
   printf("L'appel %ld donne le PID : %ld\n",SYS_getpid, mypid);
   return 0;
}
```

sudo apt update
sudo apt install build-essential
sudo apt-get install manpages-dev
gcc --version

Compilation et exécution

```
gcc gpid.c -o gpid
./gpid
L'appel 39 donne le PID : 12549
```

Prototype de la fonction main : lien

Fonctions de haut niveau

Les bibliothèques standards des langages de programmation peuvent également proposer des fonctions de haut niveau qui, au final, demandent au noyau d'effectuer différentes opérations par des appels système.

Elles assurent une meilleure portabilité, car elles sont en général indépendantes du système d'exploitation. Ainsi, un programme source utilisant les fonctions d'entrée/sorties de haut niveau du langage C pourra, sans modification, être compilé pour être exécuté sur Windows ou sur Linux.

Exemple

En langage C, on peut utiliser la fonction fopen() pour demander l'ouverture d'un fichier. Cette fonction nécessite moins d'arguments que la fonction enveloppe correspondante open(), même si les deux utilisent au final le même appel système open.

Normes et standards

Les systèmes de type Unix ont fait l'objet de différentes normalisations, dans le but de faciliter le portage des applications et l'interopérabilité des systèmes.

POSIX (Portable Operating System Interface X)

On distingue deux ensembles de normes POSIX :

POSIX.1 : définit un ensemble d'interfaces du système d'exploitation vers les applications, ainsi que les bibliothèques logicielles C permettant leur utilisation. Cet ensemble correspond aux appels système. Il y a environ 80 appels système POSIX principaux (leur nombre exact dépend des sous-catégories incluses, temps réel, réseau, etc.).

POSIX.2 : définit les caractéristiques de l'interpréteur de commandes (*shell*) ainsi qu'un jeu de commandes.

Linux implémente ces deux niveaux de normes. Il propose également des extensions qui lui sont plus ou moins spécifiques (un certain nombre de ces extensions sont communes aux différents Unix propriétaires et aux systèmes BSD). **Le noyau Linux actuel gère plus de 300 appels système**. La liste complète peut être obtenue par la commande **man 2 syscalls**.

Documentation

Les appels système, sont documentés, entre autres, dans la section 2 du manuel, accessible par la commande man.

Cette section contient une partie intro, décrivant les caractéristiques générales des appels système, et une entrée par appel système (ou par ensemble d'appels système de nature similaire). Pour chaque appel, sont spécifiés sa syntaxe générale, les fichiers d'en-tête nécessaires à son utilisation, les différents paramètres qu'il attend, les codes retours susceptibles d'être reçus et sa conformité aux différentes normes.

Les types redéfinis, déclarés dans le fichier d'en-tête sys/types.h, sont décrits dans la section 5 du manuel : man 5 types.

Les codes erreurs et les constantes symboliques qui leur correspondent, sont documentés par la commande : man errno.

Exemple

man 2 unlink

Portabilité et types

Pour renforcer la portabilité et masquer les différences d'architecture matérielle, des types redéfinis ont été inclus dans les normes.

Ils sont implémentés par des fichiers d'en-tête, le principal étant sys/types.h. Il est conseillé de les utiliser, de façon à assurer une bonne portabilité pour les programmes.

Beaucoup de ces types correspondent à des entiers. Ils permettent en particulier de contourner les problèmes liés aux différences de longueur des entiers, entre systèmes 32 bits ou 64 bits, ou en cas d'évolution des valeurs admises pour certains objets (extension d'un identifiant, initialement sur 16 ou 32 bits, en 64 bits ou davantage).

Gestion des erreurs

Presque tous les appels système retournent une valeur indiquant si l'appel a réussi ou non. Il est bien sûr très important de vérifier ce code retour, pour s'assurer que l'appel s'est déroulé correctement, sans quoi des erreurs graves et difficiles à diagnostiquer peuvent se produire à l'exécution du programme.

La documentation de chaque appel système (man 2) décrit l'ensemble des codes retours qu'il est susceptible de retourner.

Quelques appels système n'échouent jamais (getpid() par exemple, qui retourne l'identifiant du processus exécutant le programme).

Code retour et variable errno

La plupart des appels système retournent -1 en cas d'erreur et positionnent la variable globale d'erreur errno.

Cette variable est définie dans le fichier d'en-tête errno.h, qui spécifie également une série de constantes symboliques pour les différentes valeurs d'erreurs possibles, toutes commençant par la lettre E. Elles sont décrites dans la documentation, accessible par la commande man errno.

!

Un appel système réussi ne réinitialise pas la variable errno à zéro, elle conserve donc la valeur de l'erreur retournée par le dernier appel infructueux.

Quelques appels système ne retournent pas -1 en cas d'erreur. Certains peuvent retourner -1 sans qu'il s'agisse forcément d'une erreur. Dans ce cas, il faut donc, avant de les exécuter, forcer la valeur d'errno à zéro, pour pouvoir la tester ensuite et savoir si l'appel s'est bien passé ou non.

Fonctions de gestion des erreurs

Quelques fonctions de la bibliothèque du langage C facilitent la gestion des messages d'erreur :

```
perror()
#include <stdio.h>
void perror(const char *msg);
```

Cette fonction affiche le message qu'on lui passe en argument, suivi du message standard associé à l'erreur système contenue dans la variable globale errno.

```
strerror()
#include <string.h>
char *strerror(int errnum);
```

Cette fonction retourne un pointeur sur une chaîne de caractères contenant le message standard associé à l'erreur système qu'on lui passe en argument.

Fonctions de gestion des erreurs

Exemple

Ce programme, nofic.c, essaye de supprimer un fichier inexistant, échoue, et affiche le message d'erreur standard correspondant.

```c
#include <stdio.h>
#include <string.h>
// Ce programme essaye de supprimer un fichier inexistant
// et affiche le message d'erreur standard.

int main(void)
{
   // Demande de suppression d'un fichier inexistant
   int res = unlink("NoFile");
   if(res)
       perror("Erreur suppression ");
}
```

Exemple d'exécution :

```
./nofic
Erreur suppression : No such file or directory
```

Un premier programme système

Nous allons écrire un petit utilitaire, taillefic.c, qui affiche la taille en octets du fichier dont on lui passe le chemin d'accès en argument :

taillefic CheminFichier

Pour cela, nous utiliserons l'appel système stat()

Documentation de l'appel système

Pour obtenir la documentation de l'appel système, nous pouvons accéder à sa description dans la section 2 du manuel.

man 2 stat

Cette documentation indique les fichiers d'en-tête nécessaires à l'utilisation de cette famille d'appels système. Puis elle décrit chaque appel.

Un premier programme système

L'appel système stat() retourne, dans la structure de type stat dont on lui a passé l'adresse, les différentes informations concernant le fichier spécifié en argument, sans que le programme ait besoin d'ouvrir ce fichier au préalable.

Le code retour est 0 en cas de succès, -1 en cas d'erreur avec positionnement de la variable globale errno.

La taille en octets du fichier se trouve dans l'élément suivant de la structure :

```
off_t st_size; /* File size (bytes) */
```

Sur Linux, le type redéfini off_t est un entier long, défini dans sys/types.h

Un premier programme système

Programme source

Une version minimale du programme proposé, taillefic.c, pourrait donc être celle-ci :

```
#include <sys/types.h>
#include <sys/stat.h>
#include <unistd.h>
#include <errno.h>
#include <stdio.h>
// Ce programme affiche la taille du fichier
// dont le chemin d'accès est passé en argument.

int main(int argc, char *argv[])
{
   int coderet;
   struct stat infos;
   if(argc < 2)
   {
       fprintf(stderr,"%s:%s
CheminFichier\n",argv[0],argv[0]);
       return 1;
   }
   // Appel système pour obtenir les informations du fichier
   coderet = stat(argv[1],&infos);
   if(!coderet)
   {
       // La taille du fichier en octets
       // est dans l'élément st_size
       printf("%ld\n",infos.st_size);
   }
   else
   {
       // Erreur
       coderet = errno;
       perror(argv[0]);
   }
   return coderet;
}
```

Un premier programme système

Compilation et exécution

Pour compiler et générer l'exécutable, on peut utiliser la commande gcc, sans avoir à spécifier d'option particulière ni de bibliothèque :

Exemples d'exécution

```
gcc taillefic.c -o taillefic
```

Fichier existant :

```
./taillefic taillefic.c
478

echo $?
0
```

Le fichier taillefic.c fait 478 octets. Le programme a retourné zéro (stocké par le shell dans la variable $?).

Un premier programme système

Fichier inexistant :

```
./taillefic inconnu
./taillefic: No such file or directory

echo $?
2
```

Le fichier inconnu n’existe pas. Le programme affiche le message d’erreur standard associé à l’erreur 2, stockée dans errno.

Un premier programme système

Permissions d'accès insuffisantes :

```
./taillefic /var/lib/postfix/fic
./taillefic: Permission denied

echo $?
13
```

Le répertoire /var/lib/postfix est interdit en accès à l'utilisateur qui a exécuté le programme.

Les fichiers

Interface fichier universelle

Un programme peut utiliser l'appel système write() pour écrire une suite d'octets dans un fichier sur disque, vers une imprimante, l'écran d'un terminal, une connexion réseau ou vers l'entrée standard d'un autre programme. Il peut de même utiliser l'appel système read() pour lire une suite d'octets depuis un fichier sur disque, un disque entier, le clavier d'un terminal, une connexion réseau ou la sortie standard d'un autre programme.

Le concept de flot d'octets (bytestream), considérant un fichier comme une suite d'octets non structurée, donne au noyau Linux une grande souplesse pour les opérations d'entrées/sorties, et rend les applications qui l'utilisent indépendantes des caractéristiques physiques des périphériques accédés.

Les fichiers

Système de fichiers

Le noyau Linux gère une arborescence globale, organisée en répertoires qui peuvent contenir des répertoires et des fichiers. Cette arborescence a un répertoire de départ, le répertoire **racine** (*root*). Elle est constituée de différents **systèmes de fichiers** autonomes, chacun monté sur l'un des répertoires de l'arborescence globale.

Chemin d'accès d'un fichier

Un processus est toujours positionné dans un répertoire de l'arborescence du système de fichiers global : son **répertoire courant** (*current working directory*).

a. Chemin d'accès absolu ou relatif

Par conséquent, un programme peut spécifier un objet de l'arborescence, fichier ou répertoire, de deux façons :

Chemin d'accès **absolu**, par rapport à la racine de l'arborescence. Dans ce cas, le chemin d'accès commence par le caractère /. Le chemin d'accès décrit la suite de répertoires à traverser, de la racine jusqu'à l'objet spécifié, chaque répertoire étant délimité par un caractère /.

Chemin d'accès **relatif**, par rapport au répertoire courant du processus. Dans ce cas, le chemin d'accès **ne** commence pas par le caractère /. Il décrit la suite de répertoires à traverser, du répertoire courant jusqu'à l'objet spécifié, chaque répertoire étant délimité par un caractère /.

b. Répertoire courant, répertoire parent

Le répertoire **courant** d'un processus peut être désigné dans un chemin d'accès par le caractère **. (point)**.

Le répertoire **parent** d'un répertoire peut être désigné dans un chemin d'accès par la chaîne de caractères **.. (double point)**.

Les fichiers

Analyse d'un chemin d'accès

Deux fonctions de la bibliothèque standard du C facilitent l'analyse des composants d'un chemin d'accès, en permettant de séparer la partie nom de l'objet (dirname) de la partie répertoires(basename) d'accès.

Syntaxe

```
#include <libgen.h>
char *dirname(char *path);
char *basename(char *path);
```

Arguments :

path	Chemin d'accès du fichier

Valeur retournée

NULL	Erreur, code erreur positionné dans la variable `errno`.
!= NULL	Adresse d'une chaîne contenant uniquement la partie répertoires (`dirname()`), ou uniquement le nom de l'objet (`basename()`).

Les fichiers

Exemple

Ce programme, base-dir.c, affiche séparément le chemin d'accès hors nom de fichier et le nom du fichier dont on lui passe le chemin d'accès complet :

Une chaîne de caractère doit impérativement contenir un caractère spécial à la fin de la chaîne, appelé "caractère de fin de chaîne". Ce caractère s'écrit'\0'

lien

```c
#include <libgen.h>
#include <stdio.h>
#include <errno.h>
#include <string.h>
// Ce programme affiche séparément les répertoires d'accès
// et le nom du chemin d'accès passé en argument
 int main(int argc, char * argv[])
{
   char *buff;
   if(argc < 2)
   {
       printf("%s: Syntaxe : base-dir CheminFichier\n",argv[0]);
       return 1;
   }
   // Répertoires d'accès au fichier
   buff = dirname(argv[1]);
   if(buff != NULL)
   {
       printf("Chemin d'accès hors nom de fichier : %s\n", buff);
   }
   else
   {
       perror("Chemin incorrect:");
       return errno;
   }
   // Nom du fichier
   buff =  basename(argv[1]);
   if(buff != NULL)
   {
       printf("Nom du fichier : %s\n", buff);
   }
   else
   {
       perror("Chemin incorrect:");
       return errno;
   }
   return 0;
}
```

Les fichiers

Gestion des fichiers ouverts

Le noyau gère en permanence l'ensemble des fichiers ouverts par les différents processus. Pour cela, il dispose de différentes tables en mémoire, liées les unes aux autres, permettant de savoir quels fichiers de l'arborescence du système de fichiers global sont ouverts, et par quels processus.

Descripteur de fichier

- Pour pouvoir accéder au contenu d'un fichier, via un appel système, un programme doit généralement le désigner en indiquant un descripteur de fichier. Il s'agit d'un nombre positif ou nul, associé au fichier par le noyau au moment de son ouverture.

Les fichiers

Table des fichiers ouverts par le processus

Le noyau maintient pour chaque processus une table contenant la liste de ses fichiers ouverts, avec une entrée par descripteur. Quand le processus se termine, le noyau ferme automatiquement tous les fichiers associés aux descripteurs de cette table (sauf s'ils sont ouverts par d'autres processus).

Chaque entrée de la table des fichiers ouverts d'un processus contient les informations suivantes :

- Un champ d'options d'état (une seule option aujourd'hui définie sur Linux : **CLOSE_ON_EXEC**).
- Un pointeur vers l'entrée correspondante dans la table des fichiers ouverts sur le système.

Un même objet de l'arborescence du système de fichiers global peut être ouvert **plusieurs fois**, par un ou plusieurs processus. Par conséquent, plusieurs descripteurs de fichiers peuvent être associés à un même objet, chacun avec ses propres caractéristiques d'ouverture et une entrée spécifique dans la table de l'ensemble des fichiers ouverts.

Les fichiers

Table des fichiers ouverts sur le système

Le noyau gère également une table rassemblant l'ensemble des ouvertures de fichiers effectuées sur le système. Pour chaque ouverture, il y stocke les informations suivantes :

- La position courante dans le fichier.
- Les attributs d'ouverture du fichier (mode et attributs d'ouverture, éventuellement modifiés après l'ouverture).
- Un pointeur vers le numéro d'identification du fichier (son **numéro d'inode**) par rapport à son système de fichiers, associé au descripteur.
- Le nombre de descripteurs de niveau processus pointant vers ce fichier ouvert.

Les fichiers

Table des inodes ouverts

Enfin, le noyau gère une **table des inodes ouverts**, chaque entrée correspondant à un objet de l'arborescence du système de fichier global ouvert par un processus. Le numéro d'inode est une valeur numérique identifiant chaque objet d'un système de fichiers, correspondant à une entrée dans la table des inodes. Les **numéros d'inodes sont uniques** par système de fichier.

Les **éléments principaux** gérés dans cette table sont :

- Le type du fichier : Valeur numérique sur 4 bits. Ce peut être un fichier ordinaire (*regular file*), un répertoire, un lien symbolique, un tube nommé, un fichier spécial (bloc ou caractère) ou un socket.
- Les droits d'accès : Codés sur 3 groupes de 3 bits, ils déterminent les droits d'accès (lecture, écriture, exécution/accès) par catégorie d'utilisateur (propriétaire, groupe, autres).
- Le propriétaire : Le numéro d'identification (*User Id*) du compte utilisateur propriétaire du fichier.
- Le groupe : le numéro d'identification (*Group Id*) du groupe d'utilisateurs associé au fichier.
- Les dates : trois dates sont maintenues : date de création/changement d'attributs, date de dernière modification, date de dernier accès.

Les fichiers : ouverture/fermeture

Pour pouvoir accéder au contenu d'un fichier, un processus doit disposer d'un descripteur de fichier associé à ce fichier, descripteur obtenu par l'ouverture du fichier.

Appel système open()

L'appel système **open()** permet à un processus de demander au noyau l'ouverture du fichier dont il lui passe le chemin d'accès en argument. Si le fichier n'existe pas, il peut éventuellement être créé, en fonction des paramètres d'ouverture spécifiés lors de l'appel système.

Le noyau traite la demande d'ouverture, en contrôlant les droits d'accès du processus sur le fichier spécifié par rapport au type d'accès demandé (lecture, écriture, avec ou sans création).

Si l'ouverture réussit, le noyau ajoute une nouvelle entrée dans la table des fichiers ouverts du processus, dans celle des descripteurs ouverts et dans celle des fichiers ouverts (sauf si le fichier était déjà ouvert).

L'appel système retourne le descripteur de fichier correspondant.

Les fichiers : ouverture/fermeture

Syntaxe

```
#include <fcntl.h>
int open(const char *path, int oflag, mode_t mode);
```

L'appel système attend au moins deux arguments. Le troisième n'est utilisé que si le fichier doit être créé.

Arguments

path	Chemin d'accès du fichier à ouvrir. S'il s'agit d'un lien symbolique, c'est le fichier cible qui sera ouvert.
oflag	Mode et options d'ouverture.
mode	Permissions en cas de création (en combinaison avec l'umask du processus).

Valeur retournée

-1	Erreur, code erreur positionné dans la variable `errno`
>=0	Succès, descripteur associé au fichier ouvert

Les fichiers : ouverture/fermeture

Modes d'ouverture

Il y a trois modes d'ouverture de base : lecture, écriture, lecture/écriture. Chacun est identifié par une valeur entière positive ou nulle, associée à une constante symbolique :

`O_RDONLY`	Lecture seulement
`O_WRONLY`	Écriture seulement
`O_RDWR`	Lecture et écriture

Les fichiers : ouverture/fermeture

Options d'ouverture

Différentes options (*flags*) peuvent être combinées avec le mode d'ouverture, chacune identifiée par une valeur entière positive, associée à une constante symbolique. Pour les combiner avec le mode d'ouverture, on utilise un **OU** binaire (|). Voici les principales options :

O_APPEND	Écriture en fin de fichier
O_CREAT	Création du fichier
O_DIRECT	Pas d'utilisation des buffers système
O_DIRECTORY	Seulement si c'est un répertoire
O_EXCL	Avec `O_CREAT` : erreur si le fichier existe déjà
O_NOATIME	Pas de mise à jour de la date d'accès (en lecture)
O_NOFOLLOW	Ne pas suivre le lien symbolique
O_TRUNC	Tronquer le fichier à l'ouverture

Les fichiers : ouverture/fermeture

O_APPEND

Les écritures se font toujours à la fin du fichier. Cette option ne concerne que les fichiers ouverts en écriture.

O_CREAT

Si le fichier n'existe pas, il doit être créé, vide, y compris si le fichier est ouvert en lecture. Si cette option est utilisée, il faut spécifier le troisième argument définissant les droits d'accès (auxquels seront appliquées les restrictions correspondant à la valeur d'umask du processus), sans quoi ils seront définis aléatoirement.

O_DIRECT

Les accès au contenu du fichier n'utiliseront pas les caches d'entrées/sorties du noyau.

O_DIRECTORY

Retourne une erreur si le fichier n'est pas un répertoire.

O_EXCL

Utilisé avec l'option O_CREAT, provoque une erreur si le fichier existe déjà. Cette option garantit que le fichier ne sera créé que par le processus courant.

O_NOATIME (spécifique Linux)

Pas de mise à jour de la date de dernier accès pour les opérations de lecture. Le processus doit être associé au propriétaire du fichier ou à un utilisateur privilégié. Cette option a pour but de limiter le nombre d'accès disques.

O_NOFOLLOW

L'ouverture échoue si le chemin d'accès spécifié est un lien symbolique.

O_TRUNC

Si le fichier existe, il est tronqué à l'ouverture, son contenu est donc perdu. Cela se produit même si le fichier est ouvert en lecture. Le processus doit avoir les droits d'écriture sur le fichier.

Exemples d'ouvertures de fichiers existants

Lecture

Ce programme, ouvreficL.c, ouvre en lecture le fichier dont on lui passe le chemin d'accès en argument.

Exemple d'exécution :

```
ls -l fic
-rw-rw-r--. 1 pba pba 6 10 juin  15:44 fic
./ouvrefic ouvrefic.c
echo $?
0
```

Le programme a ouvert en lecture le fichier existant et a retourné la valeur 0, stockée dans la variable $? du Shell.

```c
#include <fcntl.h>
#include <errno.h>
// Ouverture en lecture du fichier
// dont on passe le chemin d'accès

int main(int argc, char *argv[])
{
    int fd;
    fd = open(argv[1], O_RDONLY);
    if(fd == -1)
    {
        perror("Open ");
        return(errno);
    }
    return 0;
}
```

ouvreficL.c

Exemples d'ouvertures de fichiers existants

Écriture

Ce programme, ouvreficE.c, ouvre en écriture le fichier dont on lui passe le chemin d'accès en argument.

Exemple d'exécution :

```
ls -l fic
-rw-rw-r--. 1 pba pba 6 10 juin  15:44 fic
./ouvreficE fic
echo $?
0
```

Le fichier a été ouvert en écriture sans erreur et n'a pas été tronqué :

```
ls -l fic
-rw-rw-r--. 1 pba pba 6 10 juin  15:44 fic
```

```c
#include <fcntl.h>
#include <errno.h>
// Ouverture en écriture du fichier
// dont on passe le chemin d'accès

int main(int argc, char *argv[])
{
    int fd;
    fd = open(argv[1], O_WRONLY);
    if(fd == -1)
    {
        perror("Open ");
        return(errno);
    }
    return 0;
}
```

ouvreficE.c

Exemples d'ouvertures de fichiers existants

Écriture avec remise à zéro

Ce programme, ouvreficEraz.c, ouvre en écriture le fichier dont on lui passe le chemin d'accès en argument, en le tronquant.

Exemples d'exécution :

```
ls -l fic
-rw-rw-r--. 1 pba pba 6 10 juin  15:44 fic
./ouvreficEraz fic
echo $?
0
```

Le fichier a été ouvert en écriture sans erreur et a été tronqué :

```
ls -l fic
-rw-rw-r--. 1 pba pba 0 10 juin  15:50 fic
```

On constate que le fichier est vide (0 octet) et que sa date de dernière modification a été mise à jour.

```c
#include <fcntl.h>
#include <errno.h>
// Ouverture en écriture du fichier
// dont on passe le chemin d'accès
// Le fichier est tronqué.

int main(int argc, char *argv[])
{
    int fd;
    fd = open(argv[1], O_WRONLY|O_TRUNC);
    if(fd == -1)
    {
        perror("Open ");
        return(errno);
    }
    return 0;
}
```

ouvreficEraz.c

Exemples d'ouvertures de fichiers existants

Lecture, sauf lien symbolique

Ce programme, ouvreficLnosym.c, ouvre en lecture le fichier dont on lui passe le chemin d'accès en argument, s'il ne s'agit pas d'un lien symbolique.

Exemple d'exécution :

Le fichier lienS est un lien symbolique, pointant vers le fichier ordinaire fichier.

```
./ouvreficLnosym lienS
Open : Too many levels of symbolic links
echo $?
40
```

L'appel système open() a retourné une erreur 40.

```c
#include <fcntl.h>
#include <errno.h>
// Ouverture en lecture du fichier
// dont on passe le chemin d'accès
// sauf si c'est un lien symbolique

int main(int argc, char *argv[])
{
    int fd;
    fd = open(argv[1], O_RDONLY|O_NOFOLLOW);
    if(fd == -1)
    {
        perror("Open ");
        return(errno);
    }
    return 0;
}
```

ouvreficLnosym.c

Exemples d'ouvertures de fichiers existants

Lecture avec remise à zéro

Ce programme, ouvreficLraz.c, ouvre en lecture le fichier dont on lui passe le chemin d'accès en argument, en le tronquant.

Exemple d'exécution :

```
ls -l fic
-rw-rw-r--. 1 pba pba 15 10 juin  16:12 fic
```

fic est un fichier ordinaire de 15 octets.

```
./ouvreficLraz fic
echo $?
0
ls -l fic
-rw-rw-r--. 1 pba pba 0 10 juin  16:15 fic
```

L'ouverture en lecture a réussi, mais le fichier a été tron modification mise à jour.

```c
#include <fcntl.h>
#include <errno.h>
// Ouverture en lecture du fichier
// dont on passe le chemin d'accès
// Le fichier est tronqué.

int main(int argc, char *argv[])
{
    int fd;
    fd = open(argv[1], O_RDONLY|O_TRUNC);
    if(fd == -1)
    {
        perror("Open ");
        return(errno);
    }
    return 0;
}
```

ouvreficLraz.c

Création d’un fichier

Avec l’option O_CREAT, l’appel système open() permet de demander la création du fichier si celui-ci n’existe pas. Dans ce cas, le noyau créera un nouveau fichier, de type « ordinaire », vide.

Le fichier aura comme propriétaire l’utilisateur associé au processus et sera associé au groupe courant du processus.

Ses permissions d’accès seront déterminées selon le troisième paramètre de l’appel open(), combiné avec la valeur d’umask du processus.

Création d'un fichier

Argument Permissions

Cet argument est un nombre spécifiant les droits d'accès à positionner, après application de l'umask.

Le nombre correspond aux 12 bits qui déterminent les droits d'accès sur un fichier Linux (4 droits spéciaux et 3 combinaisons (Propriétaire/Groupe/Autres) de 3 bits Lecture/Écriture/Exécution).

On peut utiliser les constantes symboliques suivantes, combinées par un **OU** binaire (|) pour les représenter :

Constante	Valeuroctale	Description
S_ISUID	04000	Bit `setuid` positionné
S_ISGID	02000	Bit `setgid` positionné
S_ENFMT	02000	Bit rendant les verrous impératifs
S_ISVTX	01000	Sticky bit
S_IRUSR	00400	Lecture pour le propriétaire
S_IWUSR	00200	Écriture pour le propriétaire
S_IXUSR	00100	Exécution pour le propriétaire
S_IRGRP	00040	Lecture pour le groupe
S_IWGRP	00020	Écriture pour le groupe
S_IXGRP	00010	Exécution pour le groupe
S_IROTH	00004	Lecture pour les autres
S_IWOTH	00002	Écriture pour les autres
S_IXOTH	00001	Exécution pour les autres
S_IRWXU	0700	Tous les droits pour le propriétaire
S_IRWXG	0070	Tous les droits pour le groupe
S_IRWXO	0007	Tous les droits pour les autres

Création d’un fichier

On peut aussi spécifier directement la valeur souhaitée avec une constante en octal (en la préfixant par zéro), par exemple :

0640 = Lecture/écriture pour le propriétaire, lecture pour le groupe, aucun pour les autres

Attention, cette valeur est combinée avec la valeur d’**umask** du processus. Cette dernière, héritée du processus parent ou fixée au préalable par le processus, spécifie les droits à ne pas accorder. Par exemple, une valeur d’umask de **022** signifie que les droits d’écriture ne sont pas accordés au groupe ni aux autres. Donc si on spécifie **666** comme troisième argument lors de l’appel **open()** et que le fichier doit être créé, ses droits seront **644** (666 - 022).

Création d’un fichier

Gestion de l’umask

La valeur d’umask associée au processus peut être obtenue ou modifiée par l’appel système umask().

Syntaxe

```
#include <sys/stat.h>
mode_t umask(mode_t mask);
```

L’appel système prend en argument la valeur d’umask à positionner pour le processus. Il n’échoue jamais et retourne la valeur d’umask précédente.

Création d’un fichier

Exemples de création d’un fichier

Ouverture d’un fichier en lecture, avec création obligatoire.

On souhaite donner les droits lecture/écriture à toutes les catégories (propriétaire, groupe, autres), en tenant compte de la valeur d’umask du processus.

Ce programme, creefic.c, affiche la valeur de l’umask puis crée le fichier dont on lui passe le chemin d’accès en argument, avec les droits 666 - umask.

```c
#include <fcntl.h>
#include <stdio.h>
#include <errno.h>
// Ce programme créé le fichier dont on lui passe
// le chemin d'accès, avec les droits 666 - umask.

int main(int argc, char *argv[])
{
   int fd, oflag,operm;
   int mask;
   oflag = O_RDONLY|O_CREAT|O_EXCL;
   operm = 0666;
   // Lecture de la valeur d'umask courante
   mask = umask(0);
   printf("UMASK %o \n",mask);
   // Il faut repositionner la valeur d'umask
   umask(mask);
   // Ouverture en création exclusive
   fd = open(argv[1], oflag, operm);
   if(fd == -1)
   {
       perror("Open ");
       return(errno);
   }
   return 0;
}
```

creefic.c

Création d'un fichier

Exemples d'exécution :

```
./creefic MonNouveauFichier
UMASK 2
ls -l MonNouveauFichier
-rw-rw-r--. 1 pba pba 0 10 juin  16:47 MonNouveauFichier
```

*Le fichier a été créé vide, avec lecture/écriture pour le propriétaire, lecture/écriture pour le groupe et lecture pour les autres, ce qui correspond bien à la combinaison des permissions demandées **666** moins les droits masqués par l'umask **002**, soit 664.*

Si on exécute à nouveau la commande, le fichier existant déjà, on obtient :

```
./creefic MonNouveauFichier
UMASK 2
Open : File exists
echo $?
17
```

*La création a échoué à cause de l'option **O_EXCL**.*

```c
#include <fcntl.h>
#include <stdio.h>
#include <errno.h>
// Ce programme créé le fichier dont on lui passe
// le chemin d'accès, avec les droits 666 - umask.

int main(int argc, char *argv[])
{
   int fd, oflag,operm;
   int mask;
   oflag = O_RDONLY|O_CREAT|O_EXCL;
   operm = 0666;
   // Lecture de la valeur d'umask courante
   mask = umask(0);
   printf("UMASK %o \n",mask);
   // Il faut repositionner la valeur d'umask
   umask(mask);
   // Ouverture en création exclusive
   fd = open(argv[1], oflag, operm);
   if(fd == -1)
   {
       perror("Open ");
       return(errno);
   }
   return 0;
}
```

Création d'un fichier

Ce programme, creefic666.c, affiche la valeur courante de l'umask, la force à zéro, puis crée le fichier dont on lui passe le chemin d'accès en argument, avec les droits 666.

Exemples d'exécution :

```
./creefic666 MonNouveauFichier1
UMASK précédent 2
ls -l MonNouveauFichier1
-rw-rw-rw-. 1 pba pba 0 10 juin  16:55 MonNouveauFichier1
```

*Le fichier a été créé vide, avec lecture/écriture pour le propriétaire, le groupe et les autres, ce qui correspond bien à la combinaison des permissions demandées **666**, la valeur d'umask étant 0 pendant l'exécution du processus.*

```c
#include <fcntl.h>
#include <stdio.h>
#include <errno.h>
// Ce programme créé le fichier dont on lui passe
// le chemin d'accès, avec les droits 666.
// Le fichier ne doit pas exister.
int main(int argc, char *argv[])
{
   int fd, oflag,operm;
   int mask;
   oflag = O_RDONLY|O_CREAT|O_EXCL;
   operm = 0666;
   // On force la valeur d'umask à zéro
   mask = umask(0);
   // On affiche l'ancienne valeur
   printf("UMASK précédent %o \n",mask);
   // Ouverture en création exclusive
   fd = open(argv[1], oflag, operm);
   if(fd == -1)
   {
       perror("Open ");
       return(errno);
   }
   return 0;
}
```

creefic666.c

Erreurs

Les erreurs rencontrées lors d'une demande d'ouverture de fichier peuvent être très variées, il est donc impératif de tester le code retour de l'appel. Voici une liste (non exhaustive) des erreurs possibles :

EACCES

Les permissions du processus par rapport au fichier ne permettent pas son ouverture dans le mode demandé, ou interdisent sa création. Le problème peut venir du fichier lui-même, du répertoire le contenant ou de l'un des répertoires figurant dans son chemin d'accès.

EAGAIN

Le fichier est verrouillé par un autre processus.

EDQUOT

Les quotas d'espace disque ou de nombre de fichiers sont atteints pour l'utilisateur associé au processus.

EEXIST

Le fichier existe déjà et les options O_CREAT et O_EXCL sont spécifiées.

EISDIR

Le fichier est un répertoire et on a demandé une ouverture en écriture. Aucun processus ne peut écrire dans un fichier répertoire (même avec un UID 0).

Erreurs

ELOOP

Il y a une chaîne de liens symboliques trop longue dans le chemin d'accès spécifié.

EMFILE

Le nombre maximum de fichiers ouverts autorisé au processus est déjà atteint.

ENFILE

Le nombre maximum de fichiers ouverts pour le système est déjà atteint.

ENOENT

Le fichier n'existe pas et sa création n'a pas été demandée par l'option O_CREAT.

EROFS

Le système de fichiers contenant le fichier à ouvrir en écriture (ou à créer) est monté en mode lecture seulement.

ETXTBSY

Le fichier à ouvrir en écriture est un programme en cours d'exécution, donc non modifiable.

Fermeture d’un fichier

L’appel système close() ferme le fichier dont on lui passe le descripteur. Le noyau force les écritures en cours dans le fichier, avant de libérer l’entrée correspondante dans la table des fichiers ouverts pour le processus. Cela libère également les verrous éventuellement associés à ce descripteur.

Syntaxe

```
#include <unistd.h>
int close(int fd);
```

Valeur retournée

0 Succès

-1 Erreur, code erreur positionné dans la variable errno

Lecture d’un fichier

Pour lire le contenu d’un fichier, on peut utiliser l’appel système read()

Syntaxe

```
#include <unistd.h>
ssize_t read(int fd, void *buffer, size_t count);
```

Arguments

- *Fd* *Descripteur du fichier, ouvert en lecture ou en lecture/écriture*
- *Buffer* *Adresse de la zone de stockage des octets lus*
- *Count* *Nombre maximum d’octets à lire*

Valeur retournée

-1	Erreur, code erreur positionné dans la variable errno
0	Fin de fichier
>0	Nombre d’octets effectivement lus

Lecture d’un fichier

Description

Cette fonction lit dans le descripteur le nombre d’octets réclamés en troisième argument et les place dans le buffer fourni en deuxième argument.

Si le descripteur permet le positionnement (par exemple un fichier disque), la lecture a lieu à l’emplacement indiqué par son indicateur de position. Ensuite, cet indicateur est augmenté du nombre d’octets lus.

L’appel-système renvoie le nombre d’octets lus. Si cette valeur correspond à la taille

demandée, tout s’est bien passé.

Si cette valeur est inférieure à la taille attendue mais qu’elle est positive, l’appel-système n’a pu lire qu’une partie des données voulues

Lecture de fichier : exemple

Ce programme, litfic.c, lit et affiche le contenu du fichier dont on lui passe le chemin d'accès. Il affiche ensuite la taille en octets du fichier.

Exemples d'exécution :

```
./litfic /etc/resolv.conf
nameserver 192.168.0.254
nameserver fd0f:ee:b0::1
Nb octets : 80
```

Si on lit le fichier spécial associé au terminal, /dev/tty, la lecture s'effectue en mode ligne. La saisie d'un CTRL-D au clavier, en début de ligne, provoque une fin de fichier, l'appel système read() retourne zéro au programme appelant :

```
./litfic /dev/tty
saisie ligne 1
saisie ligne 1
saisie ligne 2
saisie ligne 2
Nb octets : 30
```

Ecriture d'un fichier

L'appel système **write()** permet d'écrire dans un fichier ouvert, à partir de la position courante.

Syntaxe

```
#include <unistd.h>
ssize_t write(int fd, const void *buffer, size_t count);
```

Arguments

- fd Descripteur du fichier
- buffer Adresse de la zone de stockage des octets à écrire
- count Nombre maximum d'octets à écrire

Valeur retournée

- -1 Erreur, code erreur positionné dans la variable errno
- >=0 Nombre d'octets effectivement écrits

Ecriture d’un fichier

Description

L’appel retourne le nombre d’octets effectivement pris en compte pour l’écriture. Ce nombre n’est pas forcément égal au nombre demandé, pour différentes raisons, par exemple :

- Le système de fichiers peut être saturé.
- Le quota d’espace disque attribué à l’utilisateur associé au processus a pu être atteint.
- La taille maximale d’un fichier a pu être atteinte.
- Le noyau positionne automatiquement l’emplacement courant du descripteur utilisé après le dernier octet de l’écriture qui vient d’être effectuée.